

TRABAJO PRÁCTICO N.º 1 – Del dominio al esquema: modelo entidad-relación, modelo relacional, normalización y DDL

Parte 1 — Modelo entidad-relación

Diccionario de entidades

Relación	Cardinalidad	Participación	Justificación
Categoría – Producto	1:N	Categoría: parcial · Producto: total	R1 una categoría puede tener 0 o muchos productos; todo producto necesita pertenecer a una categoría.
Cliente – Pedido	1:N	Cliente parcial · Pedido: total	R2: un cliente puede no tener pedidos o muchos pedidos; todo pedido necesita pertenecer a un cliente.
Pedido – Producto (por Detalle_Pedido)	N:M	Pedido: total · Producto: parcial	R3 + R4: ver pregunta guía abajo

Preguntas guía

1. ¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?
2. ¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?
3. ¿Alguno de tus atributos podría descomponerse en partes más simples (por ejemplo, un nombre completo en nombre y apellido)? Justificá tu decisión de dejarlo compuesto o de separarlo.

Respuestas

1. Una relación 1:N solo admite dos posibilidades y ambas fallan:
 - **FK id_pedido dentro de PRODUCTO** (cada producto asociado a un único pedido): falla porque el mismo producto se vende en múltiples pedidos a lo largo del tiempo. Si la Muzzarella ya "pertenece" al Pedido 1, no hay dónde anotar que también aparece en el Pedido 3 y el Pedido 5 sin duplicar el producto en el catálogo
 - **FK id_producto dentro de PEDIDO** (el FK representaría un único producto asociado a cada pedido): falla porque un pedido puede combinar varios productos distintos (pizza + gaseosa). Con un solo campo de producto por pedido, no hay forma de registrar ambos sin

fragmentar la compra en pedidos separados — y para el cliente y el negocio es una sola operación, con una sola fecha y un solo total.

Como un producto puede estar en muchos pedidos y un pedido puede tener muchos productos, la relación es N:M y se resuelve creando la tabla intermedia DETALLE_PEDIDO.

Además de evitar duplicados, DETALLE_PEDIDO permite almacenar los atributos propios de cada combinación pedido-producto, como cantidad y precio_unitario. La cantidad corresponde a cuánto de ese producto se compró en ese pedido, mientras que el precio_unitario permite conservar el precio aplicado en ese momento, independientemente de posteriores cambios en el precio del producto.

2. Identificación de participaciones:

- **CATEGORIA tiene participación parcial:** puede existir sin tener productos asociados (cardinalidad mínima 0). Esto permite crear y dar de alta categorías antes de cargar los productos al inventario.
- **PRODUCTO tiene participación total:** todo producto debe estar obligatoriamente vinculado a una categoría (cardinalidad mínima 1). En el modelo relacional, esto se refleja mediante una FK id_categoria obligatoria (NOT NULL).
- Esto se desprende de R1: una categoría puede tener 0 o muchos productos, mientras que todo producto debe pertenecer a una categoría.

Consecuencias de invertir la lectura:

- **Si CATEGORIA fuera total:** no se podría crear una categoría nueva sin asignarle inmediatamente al menos un producto. Esto impediría, por ejemplo, crear categorías estacionales o preparar categorías antes de cargar sus productos.
 - **Si PRODUCTO fuera parcial:** se permitiría registrar productos sin una categoría asociada. En el modelo relacional esto podría representarse permitiendo NULL en id_categoria, generando productos sin clasificar y rompiendo la regla de negocio establecida en R1.
3. Se separan los datos, cuando sus componentes tienen significado propio para el dominio y necesitan ser consultados o manipulados de forma independiente. No es necesario descomponer un atributo simplemente porque internamente pueda contener distintas partes.
- **nombre y apellido en CLIENTE:** se almacenan como atributos separados.
Se toma esta decisión porque el dominio requiere poder utilizar cada dato de manera independiente, por ejemplo, ordenar clientes por apellido, buscar por nombre o personalizar comunicaciones. Separarlos también evita tener que interpretar posteriormente un texto completo

para determinar qué parte corresponde a cada uno, especialmente en casos de apellidos compuestos como “De la Torre”.

- **fecha en PEDIDO:** se mantiene como un único atributo de tipo DATE. Aunque una fecha contiene día, mes y año, para el dominio del negocio funciona como un único dato temporal. No hay ningún caso de uso planteado que requiera almacenar esos componentes como atributos independientes.
- **descripción en PRODUCTO:** se mantiene como un único atributo. Es información descriptiva en formato de texto y, según el dominio planteado, no es necesario consultar ni manipular partes específicas de ella como datos independientes.
- **telefono y email en CLIENTE:** se mantienen como atributos atómicos. Según las reglas del dominio, se almacenan y utilizan como datos de contacto completos, sin necesidad de acceder individualmente a partes internas de sus valores.

Parte 2 — Derivación al modelo relacional

Modelo relacional derivado: a partir del diagrama ER. La clave primaria se denota subrayada y las claves foráneas con la flecha →

categoría (id_categoria, nombre, activo)

cliente (id_cliente, nombre, apellido, email, telefono)

producto (id_producto, nombre, descripcion, precio, stock, activo, id_categoria → categoría)

pedido (id_pedido, fecha, forma_pago, id_cliente → cliente)

detalle_pedido (id_detalle, cantidad, precio_unitario, id_pedido → pedido, id_producto → producto)

Justificación de la clave primaria en la tabla intermedia

Para la tabla detalle_pedido se eligió una clave sustituta (surrogate key) propia (id_detalle), complementada con una restricción de unicidad (unique constraint) sobre la combinación (id_pedido, id_producto).

- **Simplicidad e indexación:** utilizar un identificador numérico único y simple facilita la indexación y referencia si en el futuro otras entidades necesitan relacionarse directamente con una línea de venta específica (por ejemplo, devoluciones, promociones aplicadas por ítem o auditorías).
- **Garantía de negocio:** la restricción UNIQUE (id_pedido, id_producto) mantiene la regla de negocio impidiendo que se duplique un mismo producto en distintas filas dentro de un mismo pedido.

Preguntas guía

1. ¿Qué pasaría con la integridad de tus datos si la tabla intermedia no incluyera ambas claves foráneas como NOT NULL?
2. Si el pedido pudiera existir sin ningún producto asociado (por ejemplo, un pedido recién iniciado), ¿cambia esto la participación que definiste en la Parte 1 para la relación con la tabla intermedia? Explica el impacto

Respuestas

1. **Pérdida del significado de la relación:** la tabla asociativa existe exclusivamente para materializar la intersección entre un pedido y un producto. Si id_pedido o id_producto permitieran valores NULL, podrían registrarse:
 - cantidades o precios vendidos sin un producto asociado (id_producto IS NULL);
 - ítems facturados o despachados sin pertenecer a ningún pedido (id_pedido IS NULL).

Esto generaría registros incompletos que no representan ninguna línea de pedido real, rompiendo la integridad lógica del negocio y provocando inconsistencias al calcular totales o consultar qué productos componen una compra.

2. Sí, cambia. En la Parte 1 se definió que PEDIDO tiene participación total (cardinalidad mínima 1) respecto a DETALLE_PEDIDO, asumiendo que un pedido no existe en el sistema sin al menos un ítem cargado. Si se permite registrar un pedido en estado "iniciado" o "borrador" antes de agregarle productos, la participación de PEDIDO pasa a ser parcial (cardinalidad mínima 0).

Impacto en el modelo y el sistema:

- **A nivel relacional:** acepta que en la tabla pedido existan registros cuyo id_pedido aún no tenga ninguna fila asociada en detalle_pedido.
- **A nivel de aplicación/negocio:** requiere manejar reglas de estado (por ejemplo, validar que el pedido contenga al menos un ítem antes de pasar a estados posteriores como Confirmado o Pagado), ya que temporalmente existirán pedidos sin monto total ni detalle

Parte 3 — Normalización hasta 3FN/BCNF

Paso 1: Determinación de la clave candidata

Observando la tabla desnormalizada:

- El atributo N.º pedido no identifica de forma unívoca a una fila (los pedidos 1 y 4 contienen varias líneas con distintos productos).
- La combinación (nro_pedido, producto) no se repite, ya que dentro de un mismo pedido cada producto aparece a lo sumo una vez.

Clave candidata: {nro_pedido, producto}

Paso 2 : Dependencias funcionales (DF)

Dependencia funcional	Justificación
nro_pedido → fecha	Un pedido se emite en una única fecha
nro_pedido → cliente	Un pedido pertenece a un único cliente
nro_pedido → forma_pago	Un pedido tiene una única forma de pago
producto → categoria	Cada producto pertenece a una única categoría
nro_pedido, producto → cantidad	La cantidad adquirida es específica de esa línea en ese pedido
nro_pedido, producto → precio_unitario	El precio cobrado corresponde a esa transacción histórica puntual (la Muzzarella vale \$1000 en el pedido 1 y \$1050 en el pedido 3)
cantidad, precio_unitario → subtotal	El subtotal es un valor calculado matemáticamente (cantidad × precio_unitario)

Paso 3: Verificación de 1FN

Estado: cumple 1FN. Todos los atributos contienen valores atómicos y no existen grupos repetitivos ni colecciones dentro de una misma celda — cada línea de producto vendido ya está en su propia fila.

Paso 4: Verificación de 2FN

Estado inicial: no cumple 2FN, debido a dependencias funcionales parciales sobre la clave compuesta {nro_pedido, producto}:

- nro_pedido → {fecha, cliente, forma_pago} (depende solo de una parte de la clave)
- producto → categoria (depende solo de la otra parte de la clave)

Descomposición a 2FN:

- pedido (nro_pedido, fecha, cliente, forma_pago)
- producto (producto, categoria)
- detalle (nro_pedido, producto, cantidad, precio_unitario, subtotal)

Paso 5 : Verificación de 3FN

Análisis de dependencias transitivas en cada tabla:

- **pedido:** todos los atributos dependen directa y exclusivamente de nro_pedido. Cumple 3FN.
- **producto:** categoría depende directamente de producto. Cumple 3FN.
- **detalle:** presenta una dependencia transitiva: {nro_pedido, producto} → {cantidad, precio_unitario} → subtotal. No cumple 3FN.

Corrección: como subtotal es un atributo derivado, se elimina de la estructura de almacenamiento físico para evitar redundancia e inconsistencias (se calcula al momento de la consulta con cantidad * precio_unitario).

Esquema en 3FN:

- pedido (nro_pedido, fecha, cliente, forma_pago)
- producto (producto, categoría)
- detalle (nro_pedido, producto, cantidad, precio_unitario)

Paso 6: Verificación de BCNF

Tabla	Determinante	¿Es clave/superclave?
pedido	nro_pedido	Sí — es la clave primaria
producto	producto	Sí — es la clave primaria
detalle	(nro_pedido, producto)	Sí — es la clave primaria

Conclusión: las tres tablas cumplen BCNF, ya que en cada una el único determinante presente coincide con su clave primaria — no queda ningún determinante adicional que exija una descomposición extra.

Paso 7 : Esquema relacional normalizado final

- pedido (nro_pedido, fecha, cliente, forma_pago)
- producto (producto, categoría)
- detalle (nro_pedido, producto, cantidad, precio_unitario) — con nro_pedido → pedido y producto → producto

Pregunta de integración

1. Comparé las tablas que obtuviste normalizando la planilla con las tablas que derivaste en la Parte 2 a partir de tu modelo ER. ¿A qué entidad de tu ER corresponde cada tabla? ¿Hay atributos de tu ER (por ejemplo, el mail del cliente) que no aparecían en esta planilla? ¿Qué te dice eso sobre la diferencia entre modelar a partir de un enunciado de negocio y

normalizar a partir de datos históricos ya existentes? TECNICATURA
UNIVERSITARIA EN PROGRAMACIÓN Página 5 de 8

2. El atributo subtotal de cada línea es, en teoría, calculable como cantidad × precio_unitario. ¿Conviene almacenarlo igual, o siempre recalcularlo al vuelo? ¿Esa decisión de diseño viola alguna forma normal, o es un tema distinto (rendimiento, trazabilidad histórica)? Justificá en dos o tres líneas.

Respuestas

1. Comparación con el diseño conceptual de la Parte 2

- **pedido:** equivale a la entidad PEDIDO, simplificando los datos del cliente a texto plano por el alcance acotado de esta planilla.
- **producto:** agrupa producto y categoría en una sola tabla. El proceso de normalización sobre estos datos solo exige aislar la dependencia producto → categoría, sin necesidad de crear una entidad CATEGORIA independiente. La necesidad de que CATEGORIA sea una entidad propia surge de las reglas de negocio globales del enunciado (R1: existencia de categorías sin productos; R7: baja lógica en vez de borrado físico) — información que el historial de ventas nunca tuvo motivo de registrar.
- **detalle:** mapea directamente a DETALLE_PEDIDO.
- **Atributos ausentes en los datos tabulares:** email, telefono, stock, descripcion y los flags activo. Esto refleja la diferencia entre modelar de forma *top-down* (a partir de los requerimientos globales del negocio) y normalizar de forma *bottom-up* (a partir de datos ya recolectados, que solo reflejan lo que efectivamente se registró en el pasado).

2. ¿Almacenar o recalcular subtotal?

Desde la teoría relacional, almacenarlo viola 3FN, porque introduce redundancia y riesgo de desincronización si cambian cantidad o precio_unitario. En la práctica, sin embargo, es común guardarlo igual como una decisión consciente de desnormalización controlada, por rendimiento (evitar recalcular en reportes masivos) o por trazabilidad histórica/auditoría de facturación. Si se decide dejarlo así, conviene garantizar su consistencia con columnas generadas (GENERATED ALWAYS AS) o mediante lógica transaccional estricta, dejando explícito que es un trade-off de diseño y no un descuido de normalización